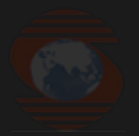
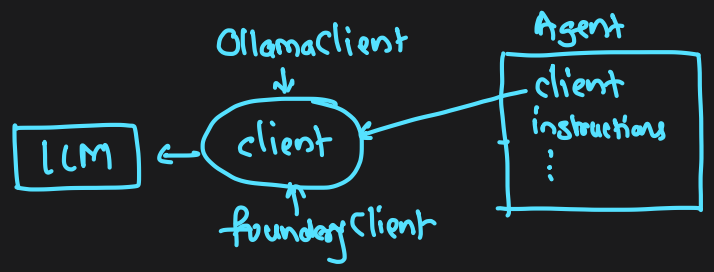
# Microsoft Agent Framework

# Introduction



- **Microsoft Agent Framework** is an open-source SDK and runtime for building AI agents and multi-agent workflows in .NET and Python
- It is the **direct successor** to Semantic Kernel and AutoGen, created by the same engineering teams
- **Why Agent Framework Exists**
  - Semantic Kernel
    - Enterprise connectors, type safety, telemetry, filters
    - Complex multi-agent abstractions
  - AutoGen
    - Simple agent abstractions, research-friendly patterns
    - Lacked enterprise state management and observability
- Agent Framework eliminates this trade-off
- It provides one framework spanning single-agent scenarios to complex multi-agent orchestration, with production-grade features built in from the start

# Core Architecture: Four Primary Areas




- **Agents**
  - Individual agents that use LLMs to process inputs, call tools and MCP servers, and generate responses
  - Supported model providers include **Microsoft Foundry, Anthropic, Azure OpenAI, OpenAI, Amazon Bedrock, Google Gemini, Ollama**, and any OpenAI-compatible endpoint
  - **Key architectural point**: All agents derive from a unified base class (Agent in Python, AIAgent in .NET), providing a consistent interface regardless of the underlying model provider
- **Harness Agent** ↳ Extensible
  - An opinionated agent with "batteries-included" capabilities for long, multi-step tasks
  - It wraps the model in a runtime that handles planning, todo tracking, context compaction, file access, memory, tool approval, and observability
- **Workflows**
  - Functional and graph-based workflows that connect agents and functions through **explicit execution paths**
  - Supports sequential, concurrent, handoff, group chat, and Magentic-One orchestration patterns
- **Integrations**
  - Connections to model providers, agent services, tools, context providers, middleware, evaluation services, and UI frameworks, organized by provider and component

# Foundational Building Blocks

- **Model clients:** Unified interface for chat completions and responses across providers
  - → Ollama | AWS Bedrock | Anthropic
- **Agent session:** Native state management for persistent multi-turn conversations → short term memory
- **Context providers:** Memory components that surface relevant information dynamically → long term mem
  - ↳ sqlite | json
- **Middleware:** Hooks to intercept, log, or modify agent actions before and after execution
- **MCP clients:** Built-in Model Context Protocol support for dynamic tool discovery at runtime
- **Function tools:** Custom functions automatically registered with schema generation handled by the framework
  - ↳ tool calling

# Basic Agent Examples



```python
from agent_framework import Agent
from agent_framework.foundry import FoundryChatClient
from azure.identity import AzureCliCredential

agent = Agent(
    client=FoundryChatClient(
        project_endpoint="https://your-foundry-service.services.ai.azure.com/api/projects/your-foundry-project",
        model="gpt-5.4-mini"
    ),
    name="HelloAgent",
    instructions="You are a friendly assistant. Keep your answers brief.",
)
result = await agent.run("What is the largest city in France?")
```

# Key Capabilities and Features



- **Multi-Turn Conversations:** Agents maintain chat history either locally or via service-managed history (Foundry, Azure OpenAI Responses, OpenAI Responses)
- **Structured Outputs:** Generate responses with type safety and schema validation
- **Streaming Responses:** Receive results incrementally as they are generated
- **Middleware:** Intercept agent actions before and after execution for logging, modification, or observability
- **MCP Support:** Built-in MCP client support enables dynamic tool discovery and integration at runtime
- **A2A Protocol:** Cross-runtime interoperability with other agent frameworks via the A2A protocol
- **Durable Workflows:** Support for long-running, human-in-the-loop scenarios with robust state management

# Agent Framework vs. Foundry Agent Service

| Consideration | Microsoft Agent Framework | Foundry Agent Service |
|---|---|---|
| **Performance** | Fast (runs locally) | Slower (managed, remote service) |
| **Development** | Full code, maximum control | Low-code, rapid integration |
| **Testing** | Manual/unit tests in code | Built-in playground for quick testing |
| **Scalability** | App-managed | Azure-managed, auto-scaling |
| **Safety Guardrails** | Custom implementation required | Built-in safety and content moderation |
| **Identity** | Custom implementation required | Built-in agent ID and authentication |
| **Enterprise** | Custom integration required | Built-in Microsoft 365/Teams deployment |

# Handling Memory



- Agent Framework distinguishes between two memory horizons:
- Short-term memory
  - Conversation history and recent context within a session
  - AgentSession + ChatMessageStore
- Long-term memory
  - Facts, preferences, and knowledge extracted across sessions
  - ContextProvider (memory providers)